Compiler

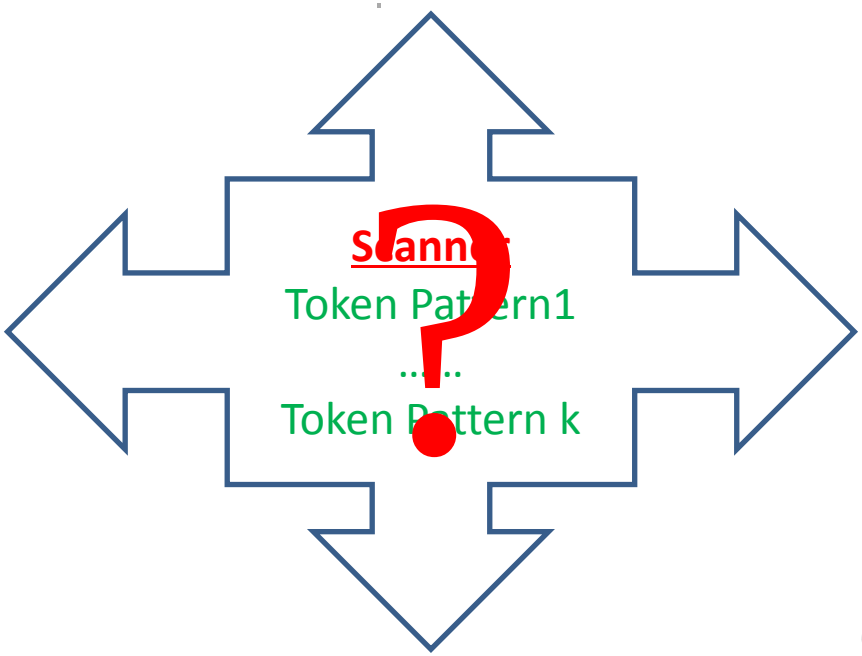
--- Lexical Analysis:

Principle&Implementation

Zhang Zhizheng
seu_zzz@seu.edu.cn
School of Computer Science and Engineering,
Software College
Southeast University

Question Addressed

```
E = M * C ** 2
```



MESSAGE

1	E	.....
2	M	.....
3	C	.....

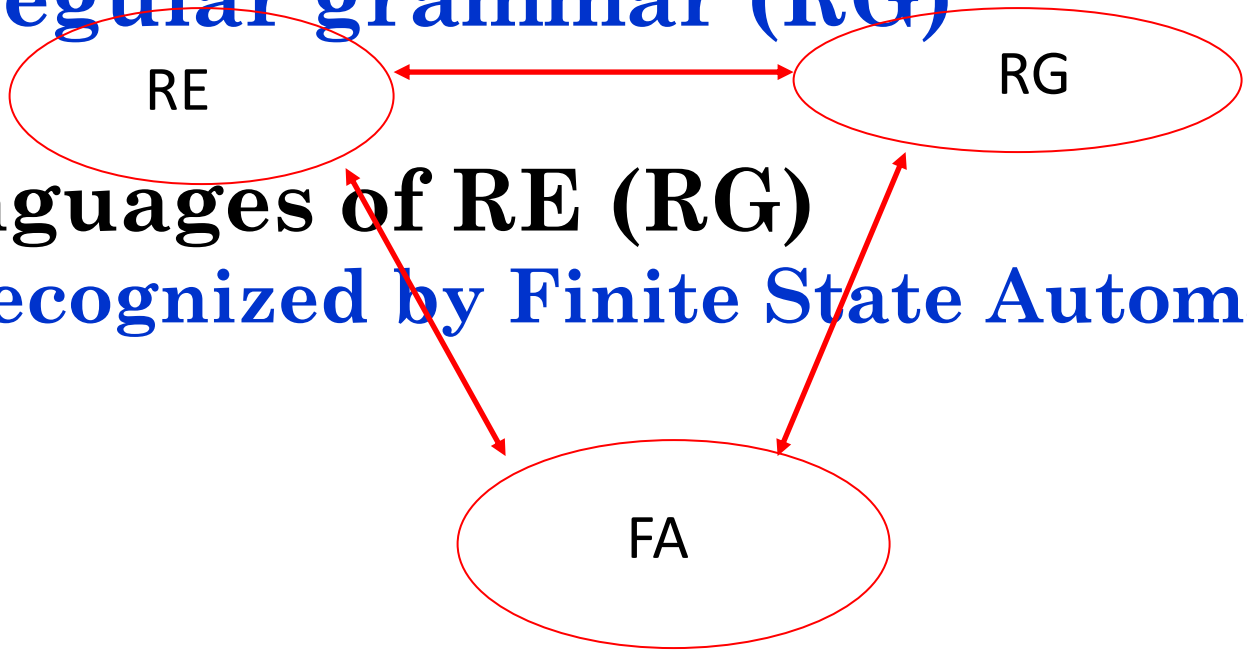
<id, pointer to symbol-table entry for E>
<assign_op>
<id, pointer to symbol-table entry for M>
<mult_op>
<id, pointer to symbol-table entry for C>
<exp_op>
<number, integer value 2>

Basic Definitions

- **Token**
  - *E.g., id, if, then, number…*
- **Token Pattern**
  - *E.g., id is a string of character and digits and begin with a character, if is 'if',……*
- **Lexeme**
  - *E.g., variable, if, then, =, ==, …, 60, 60.0*

Basic Principles I

- Token patterns are represented in
  - **Regular expressions (RE),** or equally
  - **Regular grammar (RG)**



- Languages of RE (RG)
  - **Recognized by Finite State Automata (FA)**

NOTE

- For any language $L(G)$ defined by a RG G, there exists a RE E such that $L(G)=L(E)$

- For any language define by a RG/RE, there is a FA that can recognize it.

Basic Principles II

- **Regular Grammar** – ***Right Linear Grammar***
  - $G=(V_T, V_N, P, S)$ is LLG, if every production in P is of the form
    - $A \rightarrow \alpha B$, or
    - $A \rightarrow \beta$

    where $A,B \in V_N$, $\alpha, \beta \in V_T^*$
  - *E.g.*,

    <ID>→ (a|b|c|…|A|……|Z)<REST>

    <REST>→(a|b|c|…|A|……|Z|0|……|9)<REST>

    <REST>→a|b|c|…|A|……|Z|0|……|9|ε

Basic Principles II

- **Regular Grammar –** ***Left Linear Grammar***
  - $G=(V_T, V_N, P, S)$ is LLG, if every production in P is of the form
    - $A \rightarrow B\alpha$, or
    - $A \rightarrow \beta$

    where $A,B \in V_N$, $\alpha, \beta \in V_T^*$
  - *E.g.*,

    <ID>→ <HEAD> (a|b|c|…|A|……|Z|0|…|9|ε)

    <HEAD>→ <HEAD>(a|b|c|…|A|……|Z|0|……|9)

    <HEAD>→a|b|c|…|A|……|Z|

Basic Principles III

- **Regular Expression**
  - *Definition of RE on an alphabet* Σ
    - ε is a RE, and $L(\varepsilon)$ *is* $\{\varepsilon\}$,
    - a is a RE, and $L(a)$ *is* $\{a\}$, if $a \in \Sigma$,
    - $r|s$ is a RE, and $L(r|s)=L(r) \cup L(s)$ if both r and s are REs,
    - rs is a RE, and $L(r|s)=L(r)L(s)$ if both r and s are REs,
    - r^* is a RE, and $L(r^*)=L(r)^*$ if r is a RE,
    - (r) is a RE, and $L((r))=L(r)$ if r is a RE.
  - *E.g.*, $ID=(a|b|...|Z)\ (a|b|...|Z|0|...|9)^*$

Basic Principles IV

Finite State Automata I

Deterministic FA (DFA)

DFA is a quintuple, $M(S,\Sigma,move,s_0,F)$

- S: a set of *states*
- Σ: the input symbol alphabet
- move: a transition function, mapping from $S\times\Sigma$ to S, *move(s,a)=s'*
- s_0: the *start* state, $s_0 \in S$
- F: a set of states *F* distinguished as *accepting* states, $F \subseteq S$

Basic Principles IV

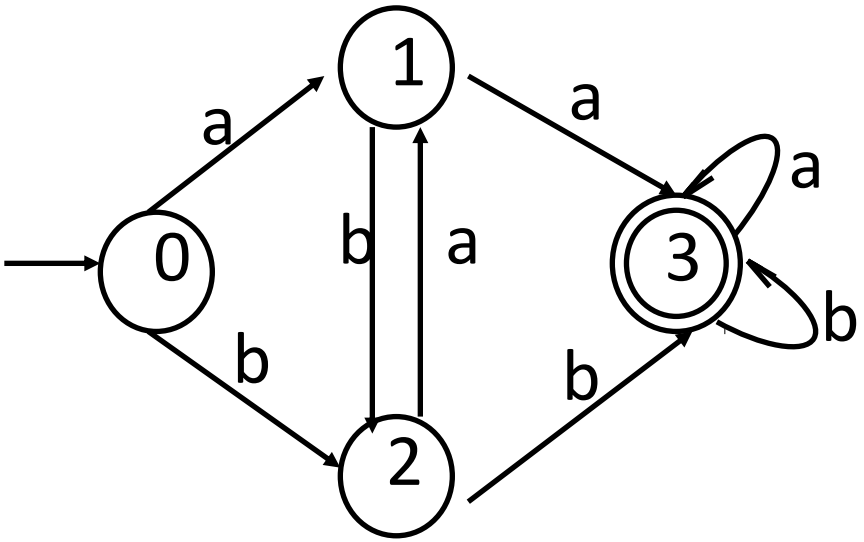
Finite State Automata II

e.g. M=({0,1,2,3},{a,b},move,0,{3})

Move: move(0,a)=1 m(0,b)=2 m(1,a)＝3 m(1,b)＝2

m(2,a)=1 m(2,b)=3 m(3,a)＝3 m(3,b)＝3

input / state	a	b
0	1	2
1	3	2
2	1	3
3	3	3

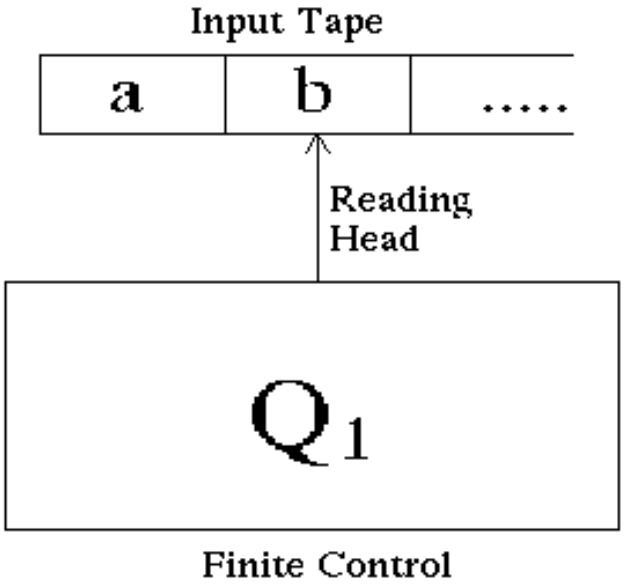


Transition graph

Basic Principles IV

Finite State Automata III

Given a DFA, the language it define is a set of strings recognized by the following equipment.

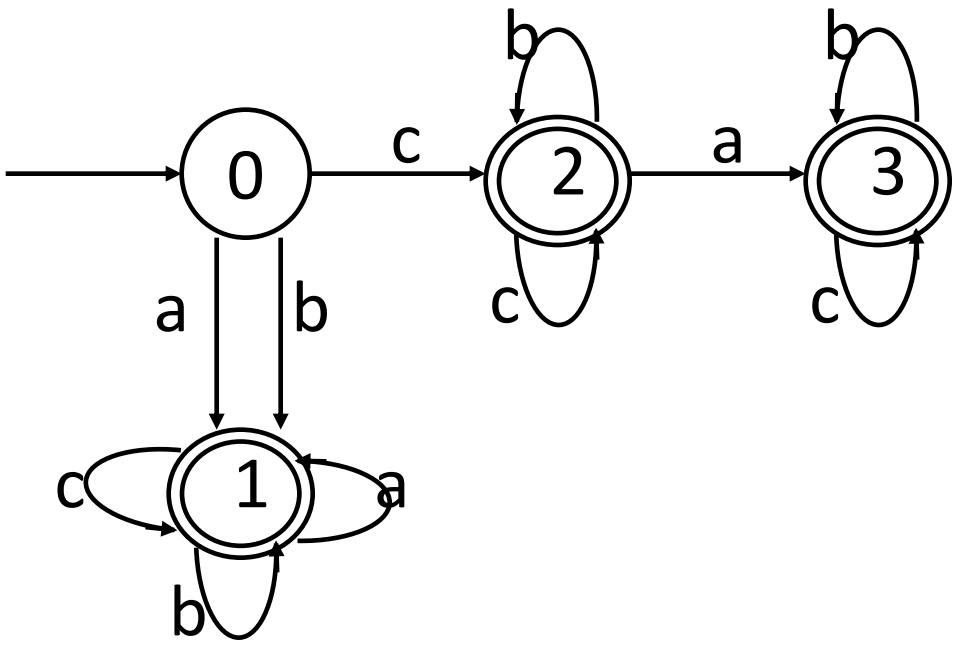


Note: **A FA accepts an input string x if and only if there is some path in the transition graph from start state to some accepting state**

Basic Principles IV

Finite State Automata IV

Please Construct a DFA M, which can accept the *a*, *b*, *c* strings which begin with *a* or *b*, or begin with *c* and contain at most one *a*. Please write a C function to implement the DFA.




```
GET()
{
    State=0;
    While(1){
        switch(state){
            case 0:  sym=nextchar()
                     if sym==a |b
                        state=1;
                     else if sym==c
                              state=2;
                      else return 0;
            case 1:  sym=nextchar()
                     if sym<>a |b|c
                        return 1;
            case 2:  sym=nextchar()
                      if sym…….
            case 3:  ……
        }
    }
}
```

Basic Principles IV

☐Finite State Automata V

Please Construct a DFA that can recognize **`ID, then`**

Please write a C function to implement the DFA.

Example1 of DFA

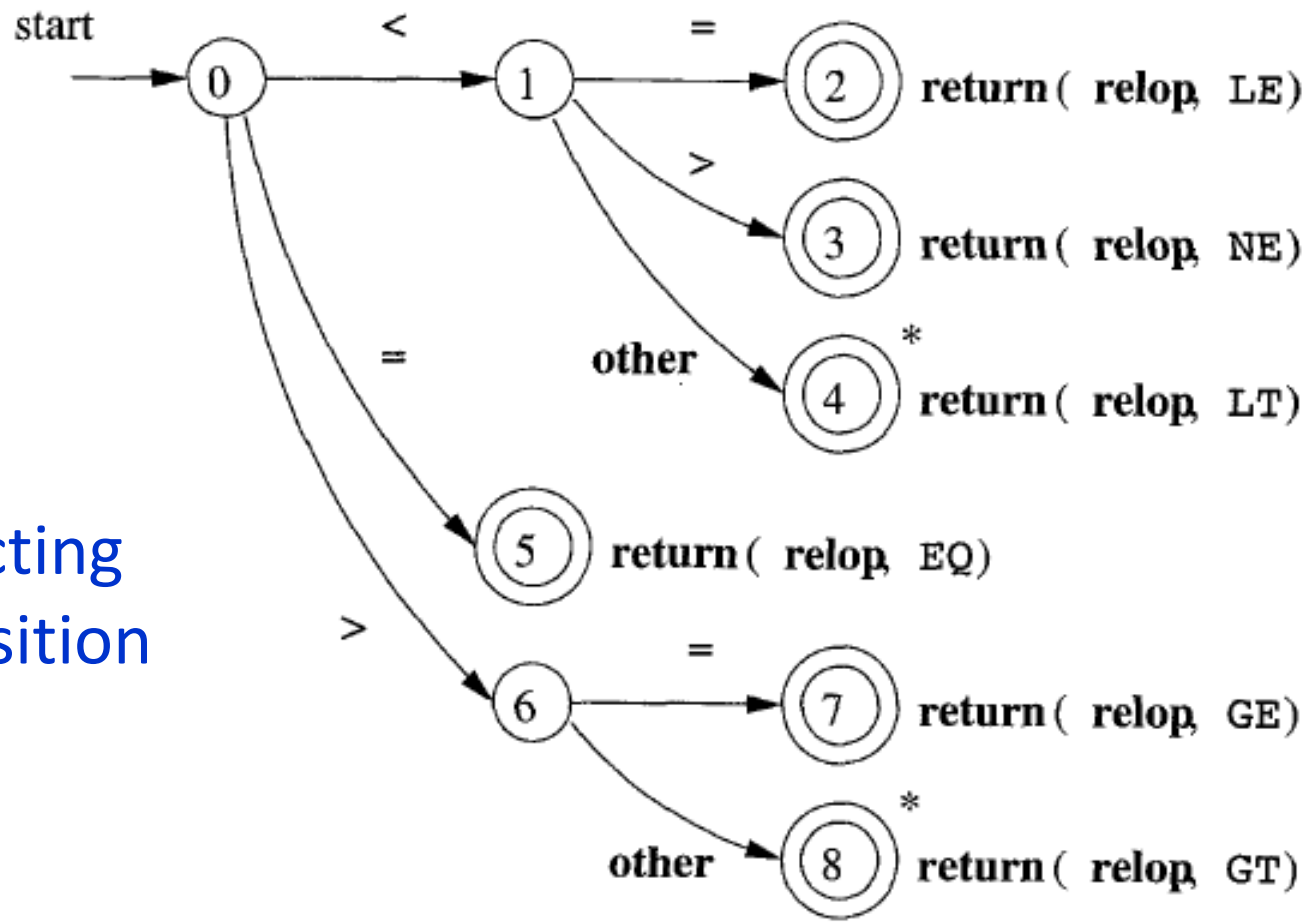


Figure 3.13: Transition diagram for **relop**

* retracting one position

Example2 of DFA

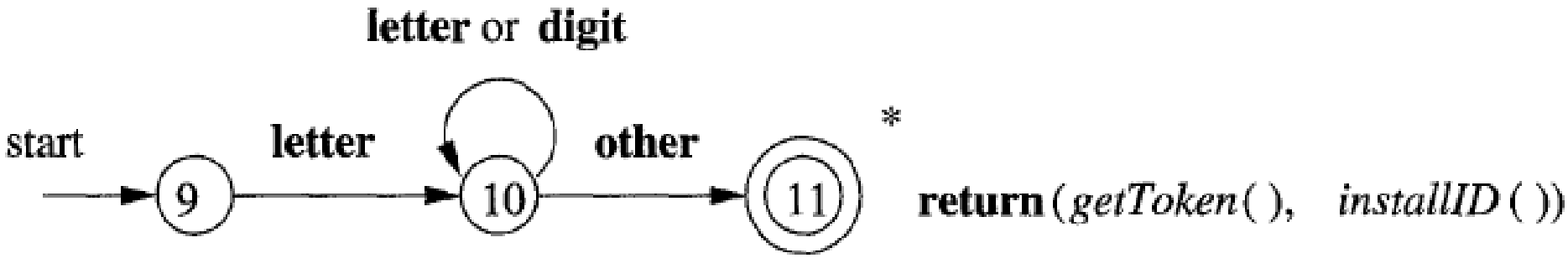


Figure 3.14: A transition diagram for **id**'s and keywords

Example3 of DFA

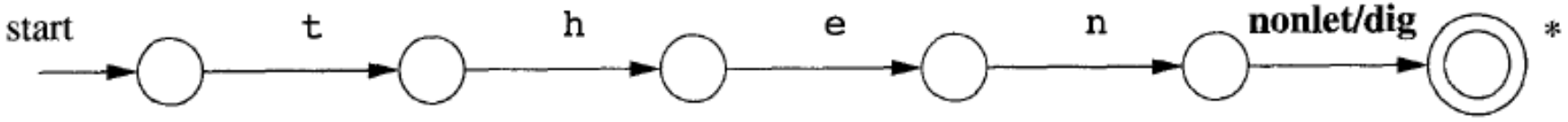


Figure 3.15: Hypothetical transition diagram for the keyword `then`

Example4 of DFA

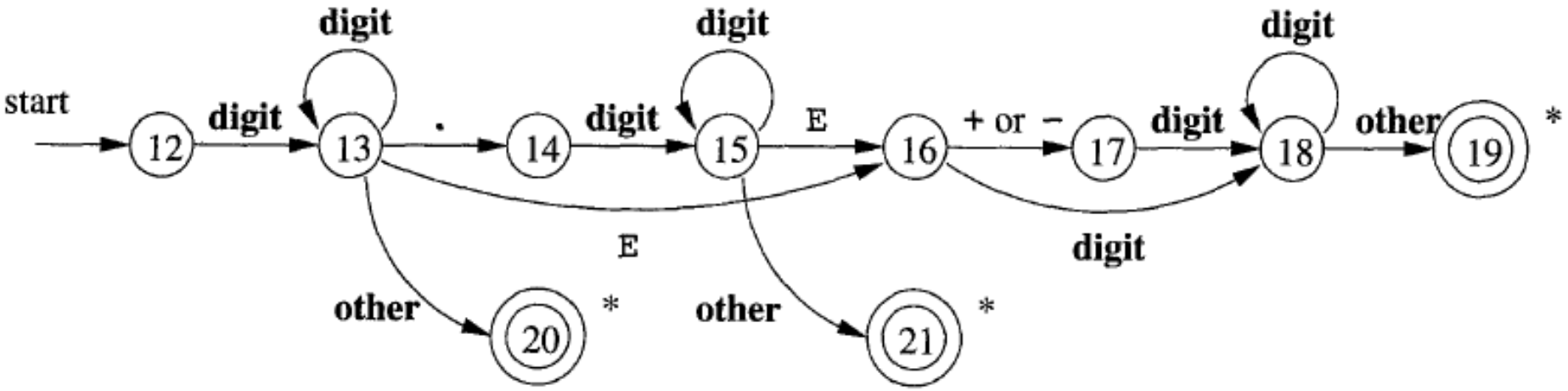


Figure 3.16: A transition diagram for unsigned numbers

Example5 of DFA

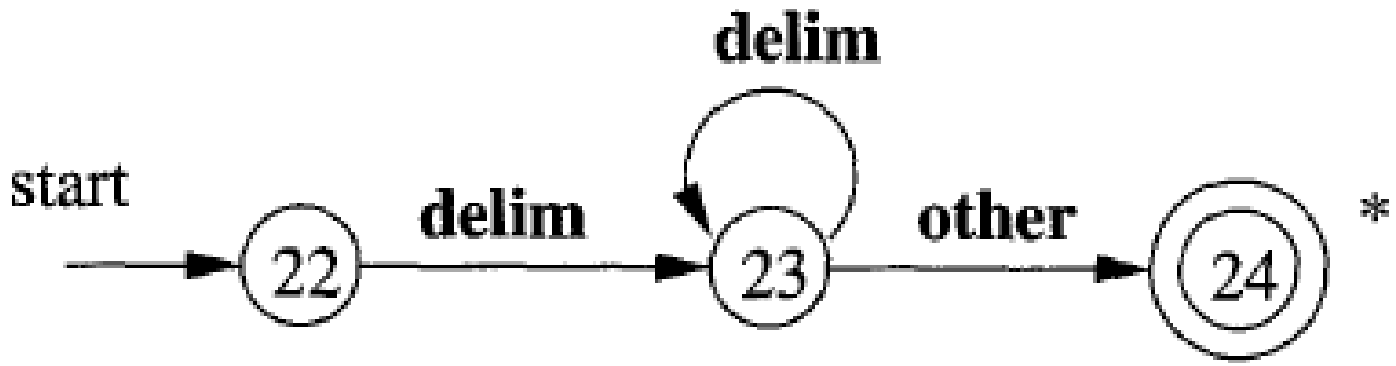


Figure 3.17: A transition diagram for whitespace

To know more details and tricks in token recognization, please read deeply in 3.4

Basic Principles IV

Finite State Automata VI

Non-deterministic FA (NFA)

NFA is a quintuple, $M(S,\Sigma,move,s_0,F)$

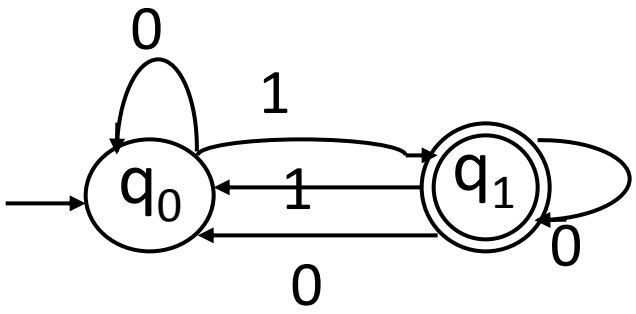
- S: a set of *states*
- Σ: the input symbol alphabet
- move: a mapping from $S\times(\Sigma\cup\varepsilon)$ to S, $move(s,a)=2^S$, $2^S \subseteq S$
- s_0: the *start* state, $s_0 \in S$
- F: a set of states *F* distinguished as *accepting* states, $F\subseteq S$

Basic Principles IV

☐Finite State Automata VII

E.g. An NFA $M=(\{q_0,q_1\},\{0,1\},move,q_0,\{q_1\})$

State \ input	0	1
q_0	q_0	q_1
q_1	$q_0,\ q_1$	q_0



Basic Principles V

- RE&RG
  - *E.g.*,

- E.g, $L=\{a^i \mid i\geq 0\}$

1) If $i=0$ $L\rightarrow\varepsilon$
2) if $i\geq 1$ $a^i = aa^{i-1}=aa^j$ $j\geq 0$ $L\rightarrow aL$
3) The grammar is as following: $L\rightarrow aL|\varepsilon$

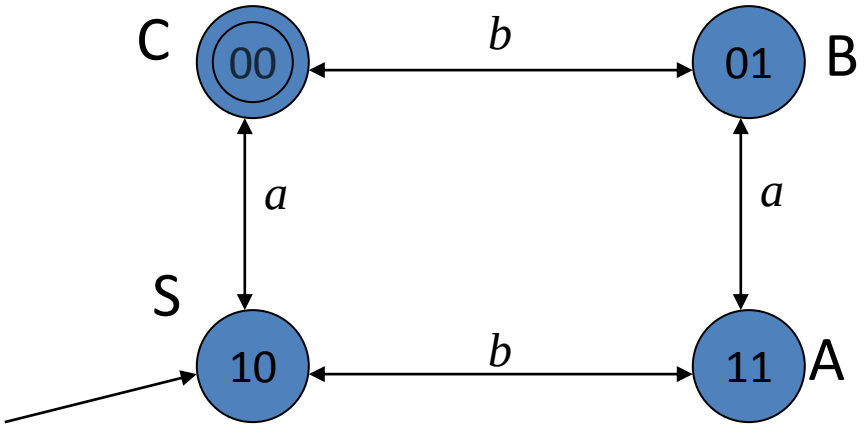
Basic Principles VI

RG&FA I

- For each regular grammar $G=(V_N,V_T,P,S)$, there is an FA $M=(Q,\Sigma,f,q0,Z)$, and $L(G)=L(M)$.
- For each FA M, there is a right-linear grammar and a left-linear grammar recognize the same language. $L(M)=L(G_R)=L(G_L)$

Basic Principles VI

- RG&FA II
  - *E.g.*,



$$S \rightarrow aC \mid bA$$
$$A \rightarrow aB \mid bS$$
$$B \rightarrow aA \mid bC$$
$$C \rightarrow aS \mid bB \mid \varepsilon$$

Basic Principles VI

- RG&FA III

  See following Algorithm of conversion from each other.

Right-linear grammar to FA

- Input :G=(V_N,V_T,P,S)
- Output : FA M=(Q, Σ,move,q_0,Z)
- Method :
  - Consider each non-terminal symbol in G as a state, and add a new state T as an accepting state.
  - Let Q=$V_N \cup \{T\}$, $\Sigma = V_T$, q_0 =S; if there is the production S$\rightarrow \varepsilon$, then Z={S,T}, else Z={T} ;

- For each production, construct the function *move*.

a) For the productions similar as $A_1 \rightarrow aA_2$, construct *move*(A_1,a)= A_2.

b) For the productions similar as $A_1 \rightarrow a$, construct *move*(A_1,a)= T.

c) For each *a* in Σ, *move*(T,a)=Φ, that means the accepting states do not recognize any terminal symbol.

E.g. A regular grammar G=({S,A,B},{a,b,c},P,S) P:

$S \rightarrow aS \mid aB$

$B \rightarrow bB \mid bA$

$A \rightarrow cA \mid c$

Construct a FA for the grammar G.

Please Construct it by yourself firstly!

Answer: let $M=(Q,\Sigma,f,q_0,Z)$

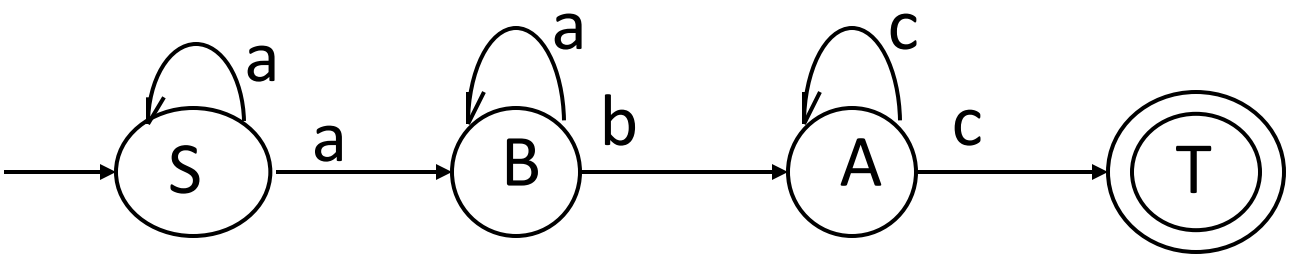
1) Add a state T , So $Q=\{S,B,A,T\}$; $\Sigma=\{a,b,c\}$; $q_0=S$; $Z=\{T\}$.

2) f：

$f(S,a)=S$ $f(S,a)=B$

$f(B,a)=B$ $f(B,b)=A$

$f(A,c)=A$ $f(A,c)=T$



FA to Right-linear grammar

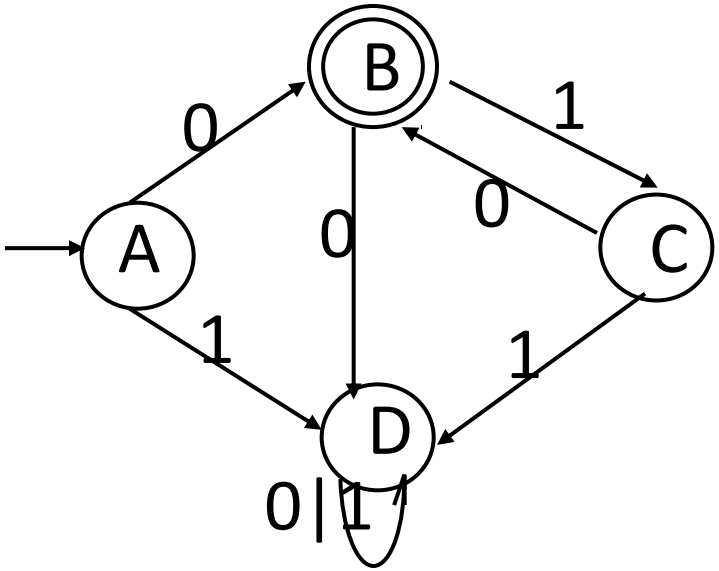
- Input : $M=(S, \Sigma, f, s_0, Z)$
- Output : $Rg=(V_N, V_T, P, s_0)$
- Method :
  - If $s_0 \notin Z$, then the Productions are;

  a) For the mapping $f(A_i,a)=A_j$ in M, there is a production $A_i \rightarrow aA_j$;

  b) If $A_j \in Z$, then add a new production $A_i \rightarrow a$, then we get $A_i \rightarrow a \mid aA_j$;

- If $s_0 \in Z$, then we will get the following productions besides the productions we've gotten based on the former rule:
- For the mapping $f(s_0,\varepsilon)=s_0$, construct new productions, $s_0' \rightarrow \varepsilon \mid s_0$, and s_0' is the new starting state.

e.g. construct a right-linear grammar for the following DFA M=({A,B,C,D},{0,1},f,A,{B})



Answer:Rg=({A,B,C,D},{0,1},P,A)

$A \rightarrow 0B \mid 1D \mid 0$

$B \rightarrow 1C \mid 0D$

$C \rightarrow 0B \mid 1D \mid 0$

$D \rightarrow 0D \mid 1D$

$L(Rg)=L(M)=0(10)^*$

Basic Principles VII

- **RE & FA**

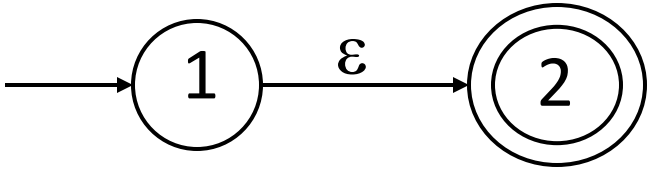
  See the following Algorithm of conversion.

Algorithm

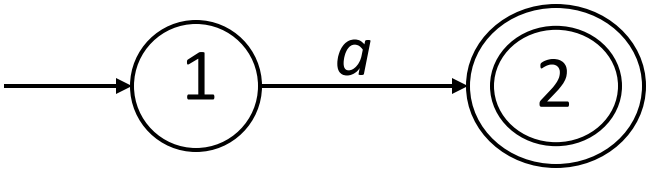
- Input. A regular expression *r* over an alphabet Σ
- Output. An NFA *N* accepting *L(r)*

Rules

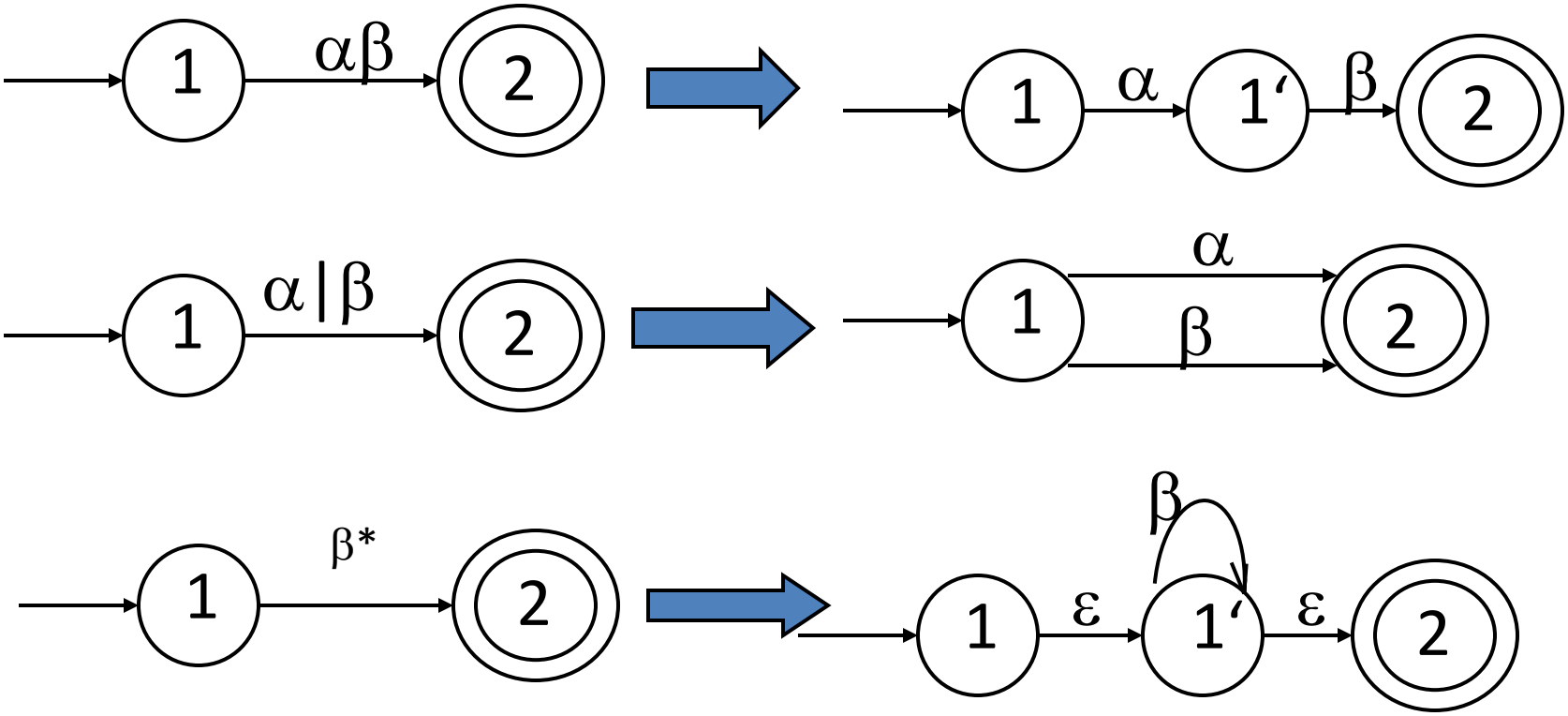
1. For ε,



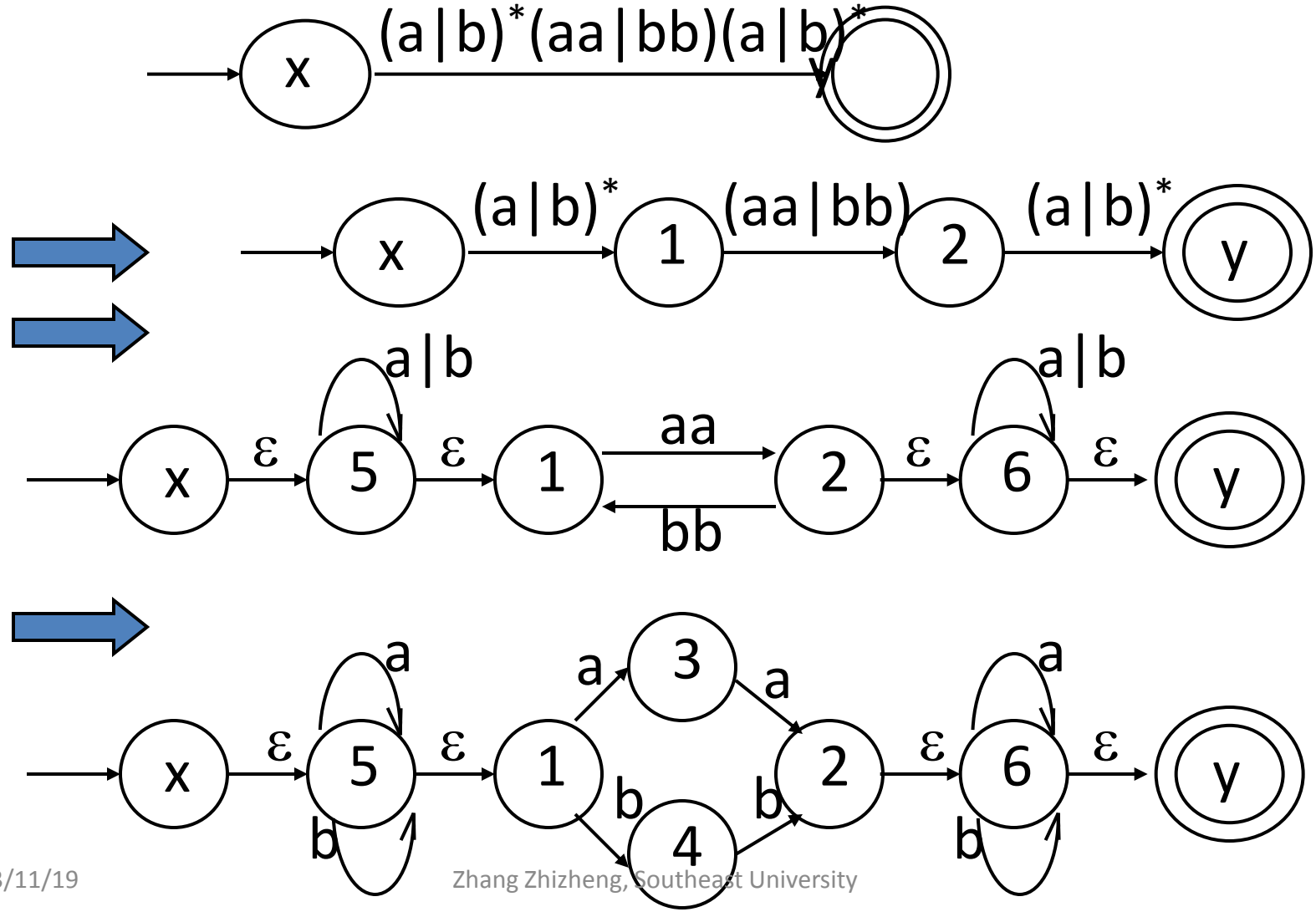
2. For a in Σ,



3. Rules for complex regular expressions



e.g. Let us construct *N(r)* for the regular expression $r=(a|b)^*(aa|bb)(a|b)^*$



Advanced Topics

- Converting NFA to DFA
- Minimizing DFA
- Merging NFAs

Advanced Topics I

Construct DFA from NFA

Find all groups of states that can be distinguished by some input string. At beginning of the process, we assume two distinguished groups of states: the group of non-accepting states and the group of accepting states. Then we use the method of partition of equivalent class on input string to partition the existed groups into smaller groups .

Advanced Topics II

☐Minimizing DFA I ---Idea

Find all groups of states that can be distinguished by some input string. At beginning of the process, we assume two distinguished groups of states: the group of non-accepting states and the group of accepting states. Then we use the method of partition of equivalent class on input string to partition the existed groups into smaller groups .

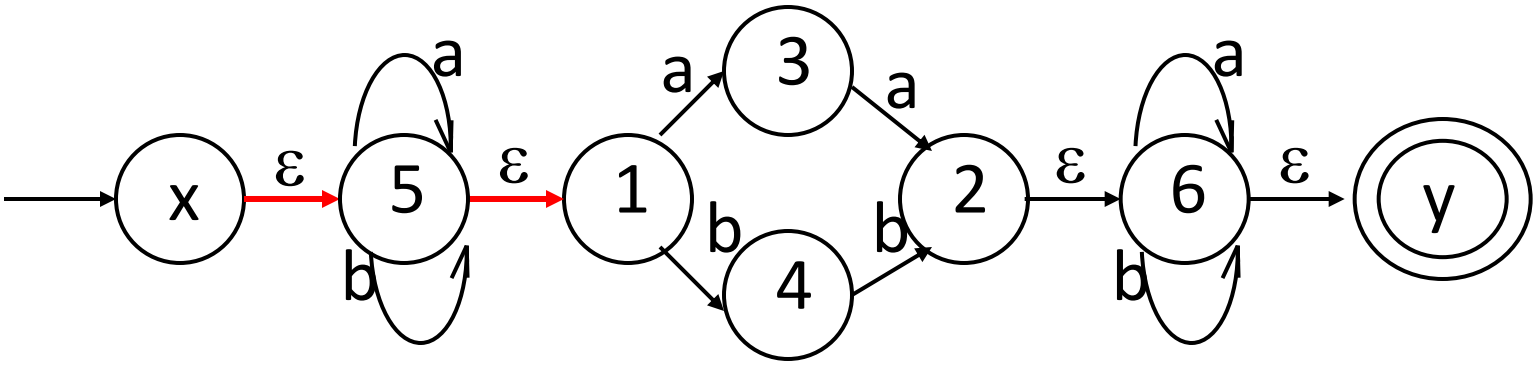
The idea of conversion algorithm

Subset construction: The following state set of a state in a NFA is thought of as a following STATE of the state in the converted DFA

Obtain ε-closure(T) $T \subseteq S$

(1) ε-closure(T) definition

A set of NFA states reachable from NFA state *s* in *T* on *ε-transitions* alone



ε -closure({x})=?

(2)ε-closure(T) algorithm

```
push all states in T onto stack;
initialize ε-closure(T)  to T;
 while stack is not empty do {
    pop the top element of the stack into t;
    for each state u with an edge from t to u labeled ε do {
     if u is not in ε-closure(T) {
        add u to ε-closure(T)
        push u into stack}}}
```

Conversion algorithm

- Input. An NFA N=(S,Σ,move,S_0,Z)
- Output. A DFA D= (Q,Σ,δ,I_0,F), accepting the same language

(1)$I_0 =$ ε-closure(S_0), $I_0 \in Q$

(2)For each I_i , $I_i \in Q$,

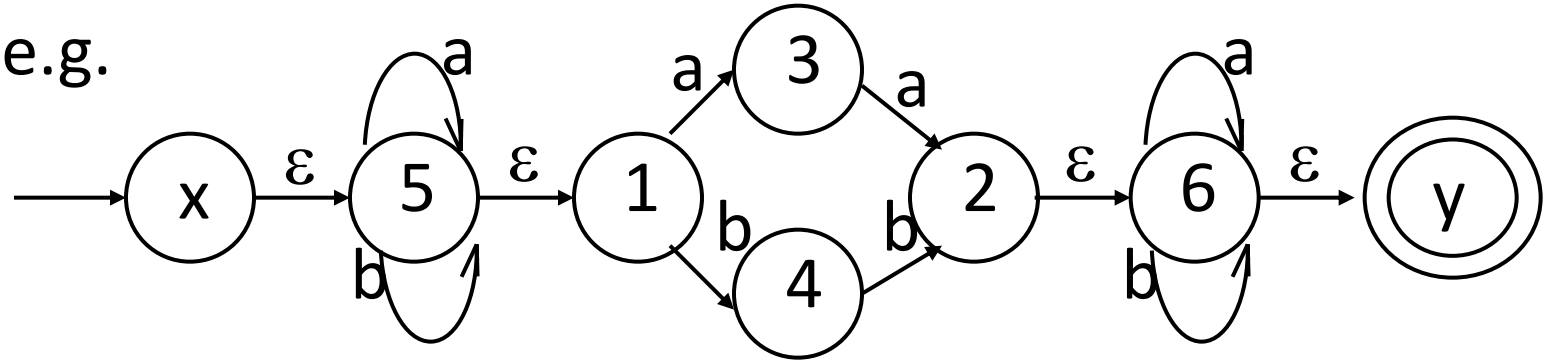
let I_t= ε-closure(move(I_i,a))

if $I_t \notin Q$, then put I_t into Q

(3)Repeat step (2), until there is no new state to put into Q

(4)Let F={I | I ∈ Q,且I ∩ Z <>Φ}

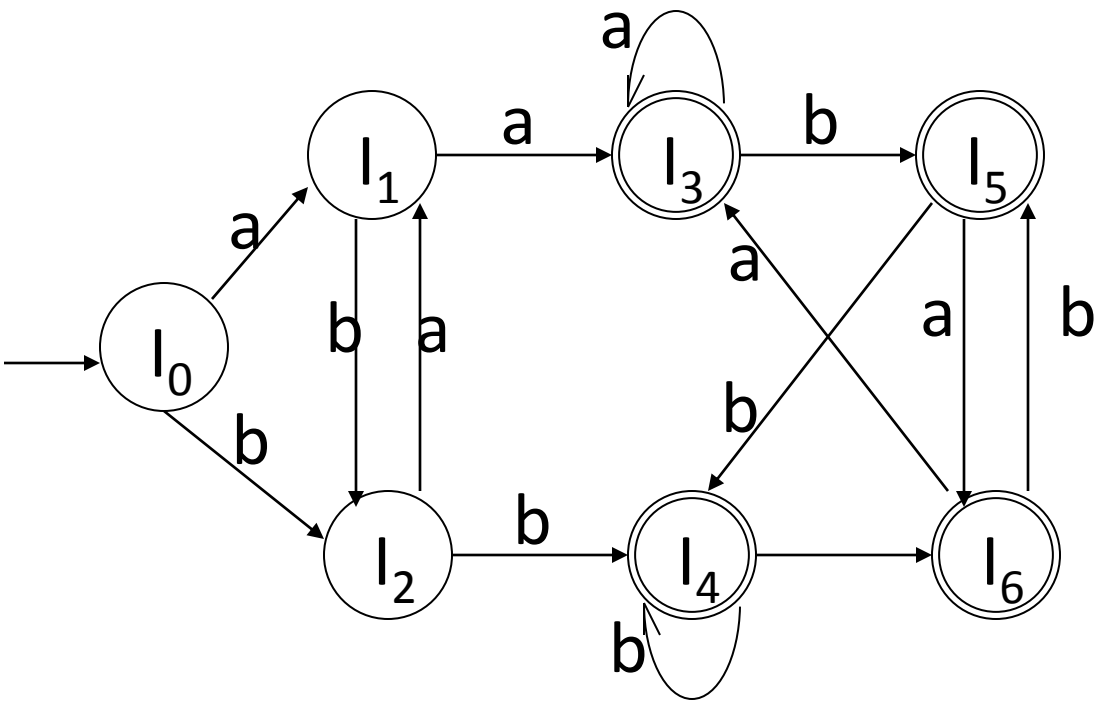
e.g.



I	a	b
I_0={x,5,1}	I_1={5,3,1}	I_2={5,4,1}
I_1={5,3,1}	I_3={5,3,2,1,6,y}	I_2={5,4,1}
I_2={5,4,1}	I_1={5,3,1}	I_4={5,4,1,2,6,y}
I_3={5,3,2,1,6,y}	I_3={5,3,2,1,6,y}	I_5={5,1,4,6,y}
I_4={5,4,1,2,6,y}	I_6={5,3,1,6,y}	I_4={5,4,1,2,6,y}
I_5={5,1,4,6,y}	I_6={5,3,1,6,y}	I_4={5,4,1,2,6,y}
I_6={5,3,1,6,y}	I_3={5,3,2,1,6,y}	I_5={5,1,4,6,y}

I	a	b
I_0	I_1	I_2
I_1	I_3	I_2
I_2	I_1	I_4
I_3	I_3	I_5
I_4	I_6	I_4
I_5	I_6	I_4
I_6	I_3	I_5

DFA is



Notes:

1) Both DFA and NFA can recognize precisely the regular sets;

2) DFA can lead to faster? recognizers

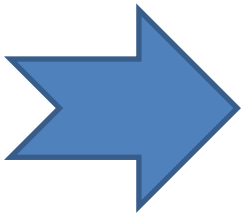
3) DFA can be much bigger than an equivalent NFA

Advanced Topics II

Minimizing DFA II—Algorithm

--Input. A DFA M={S,Σ,move, s_0,F}

--Output. A DFA M' accepting the same language as M and having as few states as possible.



Step 1. Construct an initial partition $\prod$ of the set of states with two groups: the accepting states *F* and the non-accepting states *S-F*. $\prod_0 = \{I_0^1, I_0^2\}$

Step 2. For each group *I* of Π_i ,partition *I* into subgroups such that two states *s* and *t* of *I* are in the same subgroup if and only if for all input symbols *a*, states *s* and *t* have transitions on *a* to states in the same group of Π_i ; replace *I* in Π_{i+1} by the set of subgroups formed.

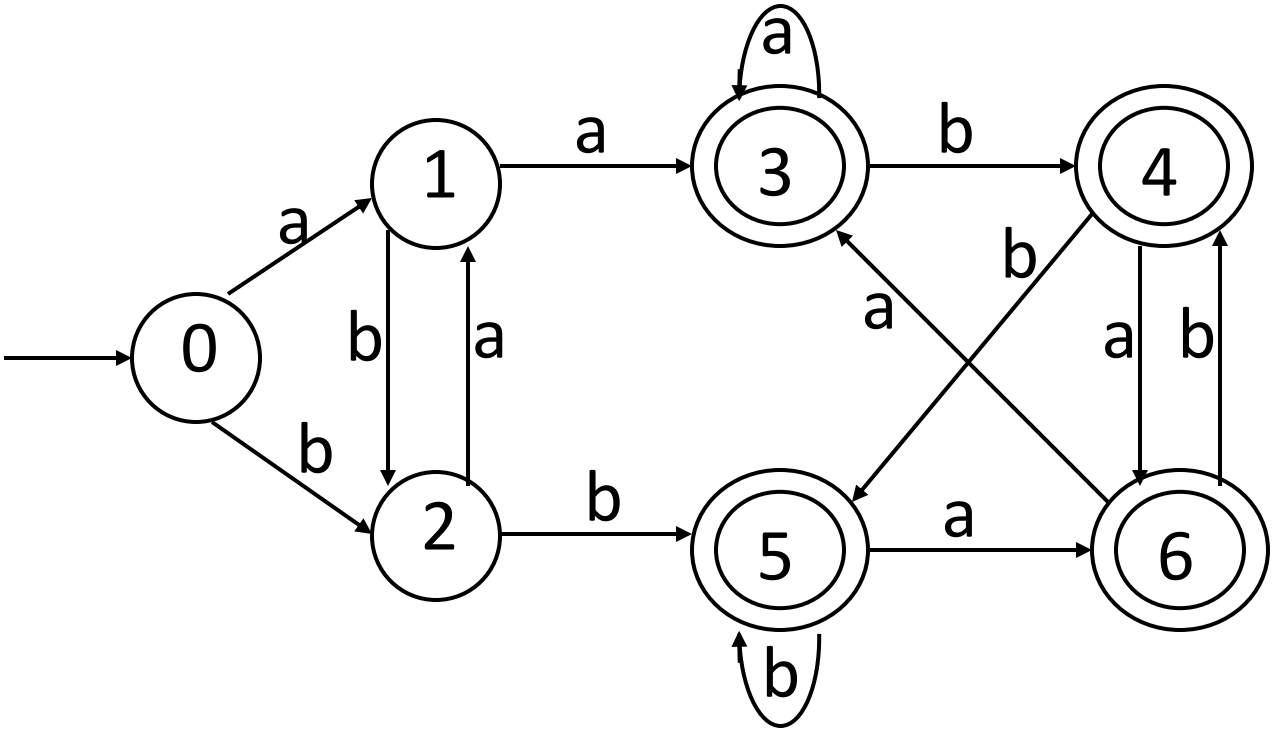
Step 3. If $\Pi_{i+1} = \Pi_i$,let $\Pi_{final} = \Pi_{i+1}$ and continue with step (4). Otherwise,repeat step (2) with Π_{i+1}

Step 4. Choose one state in each group of the partition Π_{final} as the representative for that group. The representatives will be the states of the reduced DFA M'. Let *s* and *t* be representative states for *s*'s and *t*'s group respectively, and suppose on input *a* there is a transition of *M* from *s* to *t*. Then *M'* has a transition from *s* to *t* on *a*.

Step 5. If M’ has a dead state(a state that is not accepting and that has transitions to itself on all input symbols),then remove it. Also remove any states not reachable from the start state.

Notes: The meaning that string *w distinguishes* state *s* from state *t is that b*y starting with the DFA M in state s and feeding it input w, we end up in an accepting state, but starting in state t and feeding it input w, we end up in a non-accepting state, or vice versa.

E.g. Minimize the following DFA.



- 1. Initialization: $\prod_0$＝{{0,1,2},{3,4,5,6}}
- 2.1 For Non-accepting states in $\prod_0$:
  - a: move({0,2},a)={1} ; move({1},a)={3} . 1,3 do not in the same subgroup of $\prod_0$.
  - So ,$\prod_1$`＝{{1}，{0,2}，{3,4,5,6}}
  - b: move({0},b)={2}; move({2},b)={5}. 2,5 do not in the same subgroup of $\prod_1$‘.
  - So, $\prod_1$``＝{{1}，{0}，{2}，{3,4,5,6}}

2.2 For accepting states in $\prod_0$:

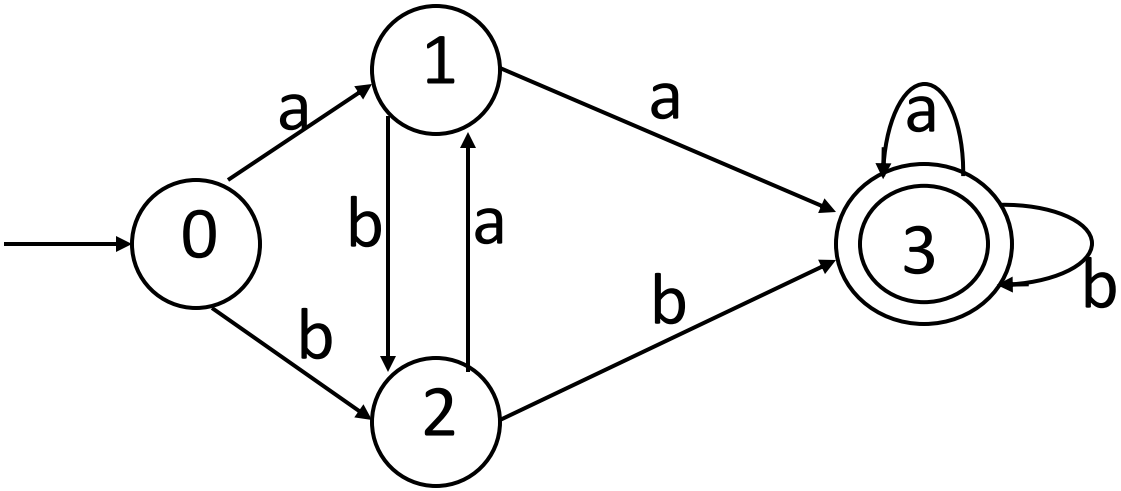
- a: move({3,4,5,6},a)={3,6}, which is the subset of {3,4,5,6} in $\prod_1$"
- b: move({3,4,5,6},b)={4,5}, which is the subset of {3,4,5,6} in $\prod_1$"
- So, $\prod_1$＝{{1}，{0}，{2}，{3,4,5,6}}.

3.Apply the step (2) again to $\prod_1$,and get $\prod_2$.

- $\prod_2$＝{{1},{0},{2},{3,4,5,6}}= $\prod_1$,
- So, $\prod_{final}$ = $\prod_1$

4. Let state 3 represent the state group {3,4,5,6}

So, the minimized DFA is :



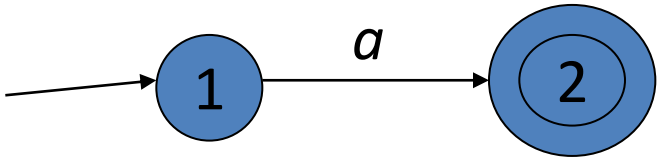
Advanced Topics III

- Merging DFAs I
  - Step 1. **Add a new start state entering each start states of DFAs by ε**
  - Step 2. **NFA → DFA**

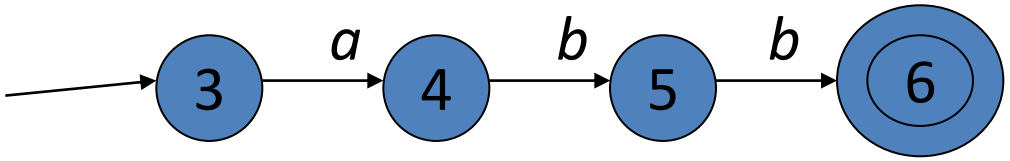
Advanced Topics III

Merging DFAs II

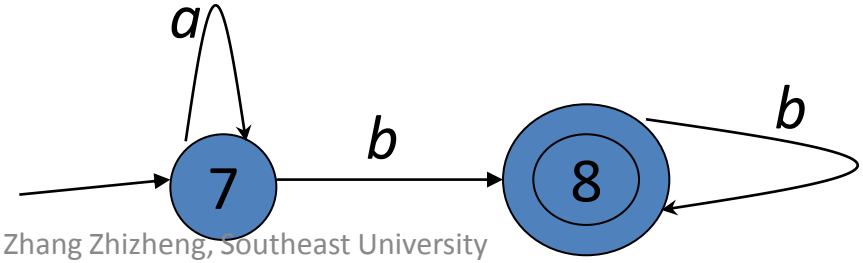
Re1 *a* ⇔



Re2 *abb* ⇔

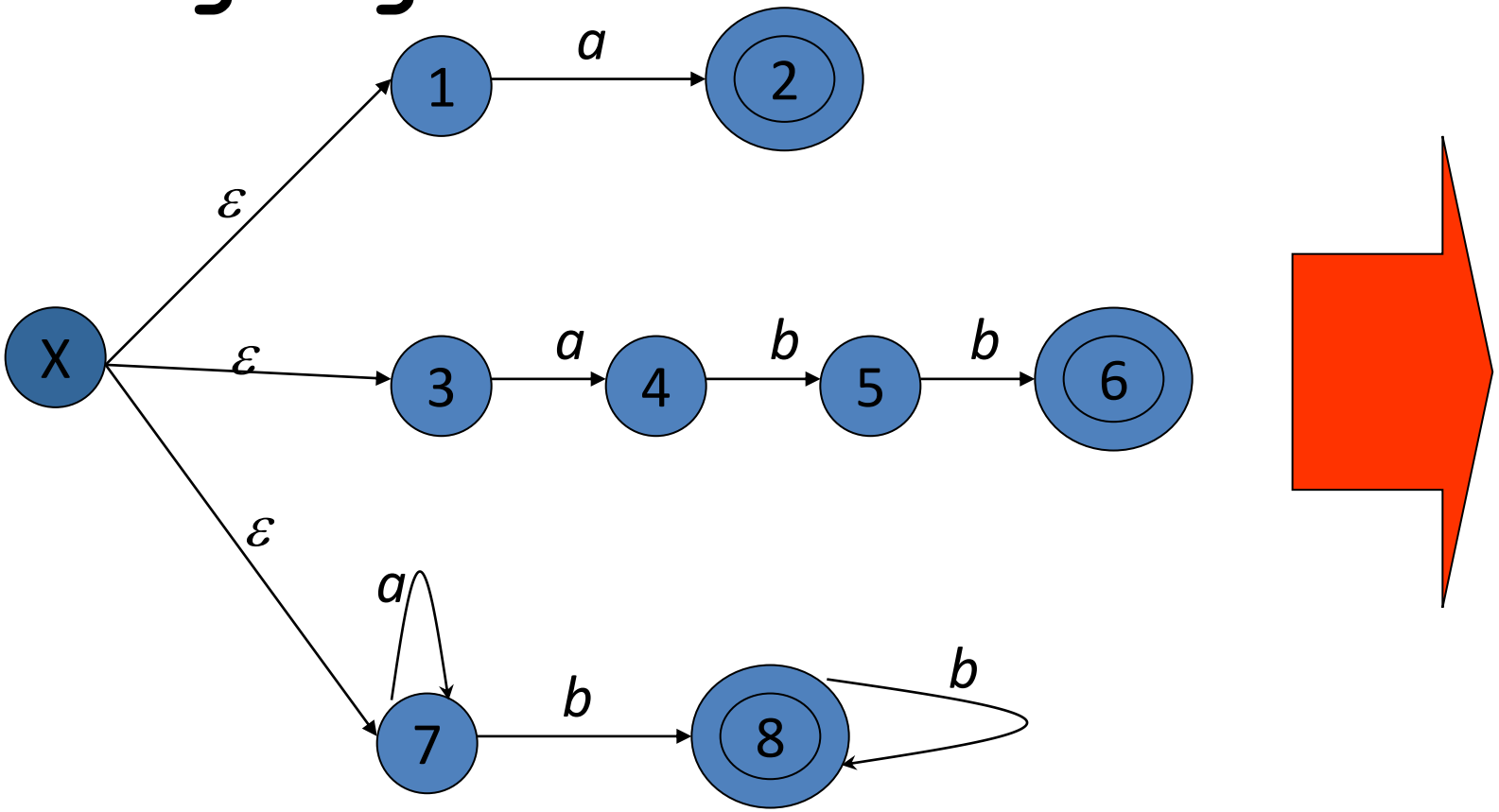


Re3 *a*bb** ⇔



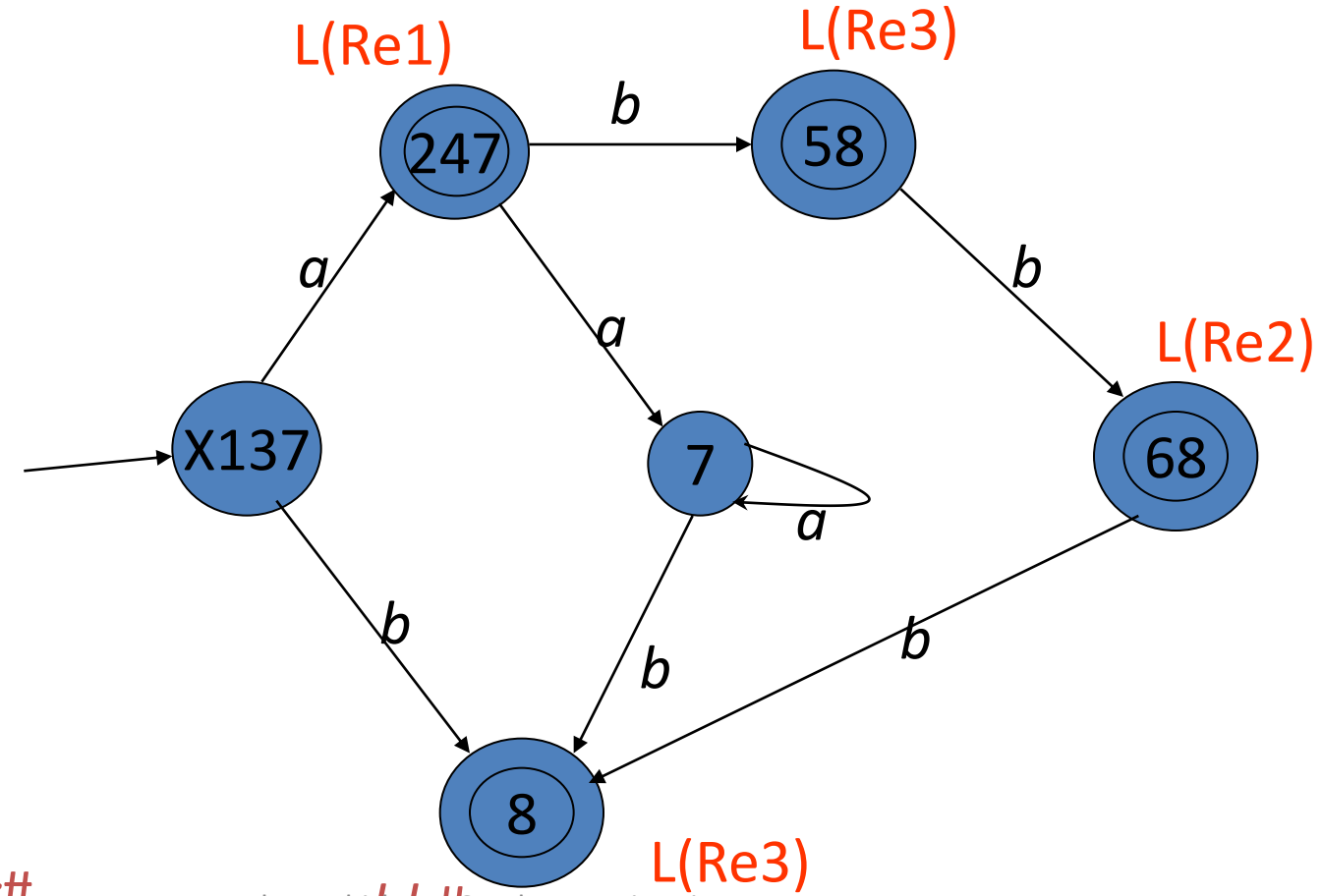
Advanced Topics III

Merging DFAs III



Advanced Topics III

Merging DFAs IV



Implementation

- Manual
- Automatic Approach by *LEX*

Implementation I

- **Manual**
  - Step 1. Designing REs
  - Step 2. Constructing NFA for each RE
  - Step 3. Merging NFAs
  - Step 4. Constructing DFA
  - Step 5. Minimize DFA
  - Step 6. Implementing DFA

Implementation II

- Automatic Approach by *LEX*

Please See 3.5

Assignments

Written

- CH3 Exercises

Programming

- See Course-Projects.ppt